1. INTRODUCTION

In today's digital age, the internet has transformed the way people access, share, and consume information. News that once traveled through newspapers, radio, and television is now available instantly on social media platforms, online portals, and mobile applications. This shift has brought unprecedented convenience, allowing billions of people across the world to remain informed in real time. However, with these advantages has come a serious challenge—the uncontrolled spread of fake news, misinformation, and disinformation. Fake news refers to deliberately fabricated or manipulated content presented as genuine news, often with the intent of deceiving, influencing public opinion, or generating financial or political gain.

The danger of fake news lies not only in its false content but also in the speed and scale at which it spreads. In earlier decades, traditional journalism was managed by editors and professional fact-checkers who ensured credibility before any information reached the public. Today, anyone with an internet connection can publish and share information instantly. Platforms like Facebook, Twitter, WhatsApp, YouTube, and Instagram allow news to go viral in seconds, reaching millions of users without any verification. The viral nature of social media has made misinformation one of the most pressing global challenges.

The consequences of fake news are wide-ranging and severe. In healthcare, the COVID-19 pandemic demonstrated how false claims regarding vaccines, treatments, and preventive measures spread rapidly, endangering public health and creating panic. In politics, fabricated stories and propaganda have influenced elections, damaged reputations, and polarized societies. At the social level, rumors spread through instant messaging platforms such as WhatsApp have triggered communal violence, mob lynchings, and widespread distrust. Economically, fake news about stock markets, companies, or financial products has misled investors and caused financial instability. These examples highlight that misinformation is not only a technological issue but also a social, psychological, and ethical problem.

The psychological nature of human behavior further complicates this problem. Research shows that people are more likely to believe and share information that aligns with their pre-existing beliefs, regardless of its factual accuracy. This phenomenon, known as confirmation bias, amplifies the spread of fake news.

Similarly, content that is emotionally charged—whether it evokes fear, anger, or excitement—tends to spread faster than neutral or factual reporting. As aresult, fake news often outperforms real news in terms of reach, visibility, and influence.

intentionally publish exaggerated or humorous content that mimics real news. Without context, AI systems and even human readers may misinterpret such articles as misinformation. Similarly, biased reporting—while based on facts— can be presented in a

way that misleads audiences. This creates a complex landscape where not every misleading piece of information can be cleanly categorized as fake.

Real-Life Examples of Fake News Impact

To highlight why such a system is necessary, let us examine a few notable real- world cases:

- **US Presidential Elections (2016, 2020):** Misinformation campaigns, including false news about candidates, had significant influence on voter perceptions. Many fabricated stories reached more people than actual verified reports.
- **COVID-19 Pandemic:** Social media platforms were flooded with misleading claims about home remedies, conspiracy theories about vaccines, and rumors about government measures. This misinformation undermined public health initiatives.
- **India's WhatsApp Rumors:** Several cases of mob violence in India were directly linked to fake news circulated on WhatsApp, especially involving child kidnapping rumors.
- **Financial Misinformation:** False tweets or articles about major corporations have led to sudden stock market fluctuations, wiping out millions in market value within hours.
- **Deepfake Videos:** AI-generated fake videos of politicians and celebrities spread rapidly, creating confusion and damaging reputations.

These examples demonstrate that fake news is not just an online nuisance—it has tangible consequences that can threaten lives, economies, and societies.

This is where the **Smart News Authenticity Checker** comes in. It builds upon the foundations of earlier fake news detection work but introduces advanced models, hybrid methodologies, and user-focused design to create a practical, real-world solution.

2. ABSTRACT

In the current digital age, the rapid expansion of the internet and social media platforms has revolutionized the way information is shared and consumed. However, this transformation has also led to a significant increase in the spread of fake news, misinformation, and disinformation. Fake news refers to false or misleading information presented as authentic news, often created with the intention of influencing public opinion, generating revenue, or spreading propaganda. The uncontrolled dissemination of such information poses serious threats to society, including political instability, public panic, and loss of trust in credible institutions.

The project titled "Smart News Authenticity Checker" aims to develop an intelligent and automated system for detecting fake news using a hybrid approach. The system integrates multiple techniques, including URL security analysis, domain credibility verification, and machine learning-based Natural Language Processing (NLP), to provide accurate and reliable results. This multi-layered approach enhances the system's ability to identify misinformation more effectively compared to traditional single-method solutions.

The system begins by analyzing the input news URL using the VirusTotal API to detect any malicious or suspicious activity associated with the link. This step ensures that harmful or phishing websites are identified at an early stage. Next, the system evaluates the credibility of the news source by comparing it with predefined databases of trusted and untrusted domains. This helps determine whether the source of the information is reliable.

The core component of the system is the content analysis module, which uses advanced NLP techniques. The textual content of the news article is extracted using web scraping tools such as Newspaper3k and processed using a pre-trained BERT (Bidirectional Encoder Representations from Transformers) model. BERT is capable of understanding the contextual meaning of words by analyzing the text in both directions, making it highly effective for classification tasks. The model predicts whether the news content is real or fake based on learned linguistic patterns and contextual features.

The system is implemented using Python, with Streamlit used to design a simple and interactive user interface. Users can input a news URL and receive real-time results categorized as Real, Fake, or Uncertain, along with additional credibility indicators. The system has been tested on standard datasets and real-world inputs, achieving an accuracy of approximately 90%.

This project provides a practical solution to combat the growing problem of fake news. It can be used by individuals, journalists, and organizations to verify the authenticity of news content. Furthermore, it promotes digital awareness and responsible information consumption. Future enhancements may include multilingual support, integration with

social media platforms, and the ability to analyze multimedia content such as images and videos.

In conclusion, the Smart News Authenticity Checker demonstrates the effectiveness of combining multiple verification techniques and machine learning models to address the challenges of fake news detection. It contributes to building a more trustworthy and reliable information ecosystem in the digital world.

3. OBJECTIVES

Primary Objective:

The primary objective of this study is to establish a comprehensive, advanced, multi-layered framework designed for the detection, analysis, and mitigation of misinformation. This framework will incorporate innovative, AI-driven solutions along with critical psychological and sociological insights, as wellas relevant policy frameworks. The study will aim to create a unified approach that addresses the challenges posed by misinformation in a digital world.

Specific Objectives:

1. Understanding the Mechanisms of Misinformation Spread:

- Evaluating Social Media Algorithms: Investigating how the algorithms of social media platforms prioritize content based on engagement rather than accuracy, contributing to the widespread dissemination of misinformation. This includes studying the amplification of sensational or misleading content that gains more clicks and interactions.
- Identifying Influential Drivers of Misinformation: Analyzing the role of various actors such as political entities, corporate interests, and ideological movements in the deliberate spread of misinformation. The research will look into how these groups utilize media channels to manipulate narratives and sway public opinion.
- Exploring Cognitive and Social Factors: Examining the psychological mechanisms, such as confirmation bias, cognitive dissonance, and echo chambers, that cause individuals to accept and believe misinformation, even in the face of contradicting evidence. The study will also explore how these factors contribute to the polarization of opinions and ideologies.

2. Assessing the Societal and Psychological Impact of Misinformation:

- Erosion of Trust in Media and Institutions: Investigating the long- term effects of misinformation on public trust in traditional media outlets governmental institutions, and other authority figures. The study will look into how misinformation weakens social and political stability by undermining trust in credible sources.
- Influence on Public Behavior: Analyzing how misinformation shapes and alters public behavior in key areas, such as public health (e.g., vaccine hesitancy), political participation (e.g., voter manipulation), and socialjustice movements (e.g., false narratives around protests). The research will assess how misinformation can have real-world consequences on public attitudes and actions.
- Role of Influencers and Public Figures: Exploring the impact of digital influencers, celebrities, and political figures in spreading or mitigating misinformation. The study

will focus on how these individuals, through their platforms, either perpetuate or challenge misinformation and influence their followers' perceptions.

3. Developing Technological Interventions:

- AI-Powered Misinformation Detection: Creating advanced misinformation detection models that utilize Natural Language Processing (NLP), sentiment analysis, and stance detection to automatically identify misleading or false content across platforms. These models will focus on real-time identification of misinformation in various forms (text, images, videos).
- Automated Fact-Checking Systems: Implementing AI-powered systems capable of conducting automated fact-checking in real-time. These systems will compare information with verified sources and flag content that misrepresents facts or provides misleading narratives.
- Predictive Models for Misinformation Spread: Developing predictive models to understand the dynamics of how misinformation propagates in different online ecosystems. This includes examining the role of network structures and the viral nature of content to predict how misinformation can spread rapidly.

4. Policy and Ethical Considerations:

- Evaluating Existing Misinformation Regulations: Conducting an in-depth evaluation of current misinformation laws and regulations across different countries to identify existing legal gaps, weaknesses, and areas for improvement in tackling misinformation while respecting free speech rights.
- Proposing New Legislative Frameworks: Developing new, innovative legislative proposals for governing misinformation, ensuring that they uphold democratic values such as free speech, while being effective in curbing harmful misinformation. This includes considering the legal boundaries of censorship and regulation in the digital age.
- Assessing Ethical Dilemmas: Analyzing the ethical implications of moderating misinformation, such as the challenges associated with censorship, the balance between free speech and harm reduction, and thepotential consequences of over-regulation on public discourse.

4. RELATED WORK

1. BERT: Pre-training of Deep Bidirectional Transformers for Language (Devlin et al., 2019)

Written by:	Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K.
Summary:	This paper introduced BERT, a transformer- based model that understands the meaning of words in context by looking at text in both directions. It became a breakthrough in NLP tasks.
Keywords:	BERT, Transformer, NLP, Contextual Embeddings
Proposed Method:	Pre-training a bidirectional transformer on a large corpus using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).
Results:	Outperformed previous models on 11 NLP benchmarks, including question answering and sentiment analysis.
Future Work:	Improve efficiency, train on multilingual datasets, and adapt for real-time applications.
Conclusion	BERT established transformers as the standard for text understanding and inspired future models like RoBERTa and XLNet.

2. Fake News Detection on Social Media: A Data Mining Perspective (Shu et al., 2017)

Written by:	Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H.
Summary:	This survey reviewed how fake news spreads on social media and explained detection approaches from a data mining perspective.
Keywords:	Fake News, Data Mining, Social Media, Detection

Proposed Method:	Categorized detection into two types—content- based and context-based. Suggested hybrid approaches.
Results:	Highlighted strengths and weaknesses of both approaches; showed context (like user behavior) improves detection.
Future Work:	Use advanced deep learning, include temporal and network features.
Conclusion	Social media fake news detection must go beyond text and include context such as user engagement and sharing patterns.

3. Lazer, D. M. J., Baum, M. A., Benkler, Y., et al. (2018). The Science of Fake News. Science, 359(6380), 1094–1096.

Written by:	Lazer, D. M. J., Baum, M. A., Benkler, Y., et al.
Summary:	This article explained how fake news spreads, why people believe it, and what challenges exist in combating it. It called for collaboration between computer scientists, journalists, and policymakers.
Keywords:	Fake News, Social Media, Public Policy, Trust
Proposed Method:	Analyzed social and psychological factors behind misinformation instead of just technical solutions.
Results:	Showed that fake news spreads faster than real news because it is more novel and emotional.
Future Work:	Create hybrid solutions combining technical AI tools with educational and regulatory strategies.

Conclusion	Fake news is a multidisciplinary problem requiring technical, social, and policy-based solutions.

4. Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake News Detection on Social Media: A Data Mining Perspective. SIGKDD Explorations, 19(1), 22–36.

Written by:	Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H.
Summary:	Explores fake news detection using content, user, and network features. Highlights the importance of data mining approaches. Integrates multiple perspectives for accurate detection.
Keywords:	Fake News Detection, Social Media, Data Mining, Machine Learning, Features
Proposed Method:	Combines textual analysis, user behavior, and network structures for classification. Machine learning models are trained on labeled data. Multi-feature integration improves performance.
Results:	Outperforms content-only models. Reduces false positives and enhances generalization. Identifies influential spreaders of fake news.
Future Work:	Study temporal patterns and cross-platform detection. Explore deep learning for richer feature extraction. Improve real-time detection scalability.
Conclusion	Integrating multiple features enhances fake news detection. Data-driven approaches provide robust monitoring. Multi-perspective analysis is essential.

5. Perez-Rosas, V., Kleinberg, B., Lefevre, A., & Mihalcea, R. (2018). Automatic Detection of Fake News. ACL Workshops, 339–349.

Written by:	Pérez-Rosas, V., Kleinberg, B., Lefevre, A., & Mihalcea, R. (2018)
Summary:	Proposes automated detection of fake news using linguistic and stylistic features. Distinguishes deceptive content from credible news. Uses NLP techniques for analysis.
Keywords:	Fake News Detection, NLP, Automatic Detection, Text Analysis, Classification
Proposed Method:	Extracts lexical, syntactic, and semantic features. Applies machine learning classifiers for prediction. Emphasizes feature engineering for reliability.
Results:	Achieves high accuracy across datasets. Detects subtle differences in fake and real news. Outperforms heuristic baselines.
Future Work:	Integrate social context and user behavior. Expand to multilingual datasets and real-time detection. Enhance robustness and scalability.
Conclusion	Linguistic and stylistic analysis effectively detects fake news. Provides a benchmark for future automated methods. Enhances reliability in misinformation detection.

6. Popat, K., Mukherjee, S., Strötgen, J., & Weikum, G. (2018). Declare: Debunking Fake News and False Claims using Evidence-Aware Deep Learning. SIGIR, 555–564.

Written by:	Popat, K., Mukherjee, S., Strötgen, J., & Weikum, G. (2018)
Summary:	DECLARE detects fake news using evidence- aware deep learning. Verifies claims against trusted sources for credibility. Integrates textual and external evidence.

Keywords:	Fake News Detection, Evidence-Aware, Deep Learning, Fact-Checking, Verification
Proposed Method:	Uses attention mechanisms to align claims with external evidence. Classifiers predict true, false, or unverifiable claims. Combines multiple information sources.
Results:	Improves detection accuracy compared to content-only models. Reduces false positives and increases reliability. Validated on benchmark datasets.
Future Work:	Explore cross-domain evidence and real-time verification. Expand multilingual and large-scale deployment.
Conclusion	Evidence-aware deep learning improves fake news detection. Bridges automatic detection and fact-checking. Reliable for misinformation verification.

7. Zhou, X., & Zafarani, R. (2020). A Survey of Fake News: Fundamental Theories, Detection Strategies and Challenges. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 4(1), Article 33.

Written by:	Zhou, X., & Zafarani, R. (2020)
Summary:	Reviews the landscape of fake news research, including definitions, theories, and detection methods. Highlights challenges like data scarcity, social context, and evolving misinformation tactics. Emphasizes the need for multi-disciplinary solutions.
Keywords:	Fake News, Detection, Social Media, Misinformation, Survey, Challenges

Proposed Method:	Categorizes existing detection strategies into content-based, context-based, and hybrid approaches. Summarizes datasets, models, and evaluation metrics. Identifies research gaps.
Results:	Provides a comprehensive understanding of detection methods and their limitations. Helps researchers select appropriate approaches. Highlights gaps in cross-domain and multilingual detection.
Future Work:	Develop real-time, explainable, and robust detection frameworks. Address multilingual and cross-platform challenges. Integrate social context and user behavior in models.
Conclusion	Fake news detection requires a combination of methods and interdisciplinary approaches. Survey guides researchers to improve detection systems and address societal challenges.

8. Wani, M. F., & Kumar, A. (2021). COVID-19 Fake News Detection Using Machine-Learning Techniques: A Systematic Review. Expert Systems with Applications, 185, 115695.

Written by:	Wani, M. F., & Kumar, A. (2021)
Summary:	Reviews machine learning techniques used to detect COVID-19 misinformation. Examines datasets, models, and evaluation metrics. Highlights challenges posed by rapid information spread during pandemics.
Keywords:	COVID-19, Fake News Detection, Machine Learning, NLP, Social Media
Proposed Method:	Summarizes supervised, unsupervised, and hybrid approaches for detecting health misinformation. Focuses on textual and social features.

Results:	ML models like SVM, Random Forest, and deep learning achieve good performance on COVID- 19 datasets. Content combined with network features improves accuracy.
Conclusion	Machine learning is effective for detecting COVID-19 fake news. Integrating content and context ensures better detection and public safety.

9. Ruchansky, N., Seo, S., & Liu, Y. (2017). CSI: A Hybrid Deep Model for Fake News Detection. ACM CIKM, 797–806.

Written by:	Ruchansky, N., Seo, S., & Liu, Y. (2017)
Summary:	CSI is a hybrid model combining content, social, and user signals to detect fake news. Leverages deep learning and temporal propagation patterns. Improves robustness over single-feature models.
Keywords:	CSI, Fake News Detection, Deep Learning, Social Signals, User Behavior
Proposed Method:	Integrates textual features with user credibility and news propagation patterns. Employs LSTM and attention mechanisms for learning complex relationships.
Results:	Outperforms content-only and shallow models. Detects fake news early in propagation. Enhances reliability of automated detection.
Future Work:	Explore cross-domain adaptability and multilingual performance. Integrate explainability for user trust.
Conclusion	CSI demonstrates the power of multi-feature, hybrid approaches. Combining content, social, and temporal signals enhances detection accuracy.

10. Zhang, X., & Ghorbani, A. A. (2020). An Overview of Online Fake News: Characterization, Detection, and Discussion. Information Processing &Management, 57(2), 102025

Written by:	Zhang, X., & Ghorbani, A. A. (2020)
Summary:	Reviews online fake news characteristics, detection techniques, and challenges. Discusses user, content, and network behaviors in dissemination. Highlights gaps in current approaches.
Keywords:	Fake News, Online Media, Detection, Social Networks, Survey
Proposed Method:	Categorizes existing detection strategies into content-based, context-based, and hybrid approaches. Summarizes datasets, models, and evaluation metrics. Identifies research gaps.
Results:	Identifies limitations in single-feature models. Hybrid approaches are more accurate. Highlights the importance of social context and verification.
Future Work:	Develop real-time, cross-domain, and multilingual detection systems. Address misinformation in emerging social media platforms.
Conclusion	Understanding fake news characteristics is critical for effective detection. Hybrid, multi- feature methods offer the most promise.

11. Shu, K., Cui, L., Wang, S., Lee, D., Liu, H., & Guo, S. (2018). Defend: Explainable Fake News Detection. SIGIR, 235–244.

Written by:	Shu, K., Cui, L., Wang, S., Lee, D., Liu, H., & Guo, S. (2018)

Summary:	DEFEND focuses on explainable fake news detection. Provides rationale for predictions using content and social features. Improves user trust and transparency.
Keywords:	DEFEND, Explainable AI, Fake News Detection, Social Features, NLP
Proposed Method:	Combines textual analysis with social propagation patterns. Generates human- understandable explanations alongside predictions.
Results:	Accurate detection while providing insights for decision-makers. Reduces reliance on opaque models.
Future Work:	Expand explainability to cross-platform and multilingual datasets. Integrate real-time monitoring capabilities.
Conclusion	DEFEND demonstrates that explainable AI improves trust in fake news detection. Transparency is key for societal adoption.

12. Potthast, M., Kiesel, J., Reinartz, K., et al. (2018). Fake News — Back to the Beginning. ECIR Workshops, 67–79.

Written by:	Potthast, M., Kiesel, J., Reinartz, K., et al. (2018)
Summary:	Explores historical patterns of fake news and initial detection efforts. Emphasizes the role of linguistic, stylistic, and propagation features. Provides a baseline for modern approaches.
Keywords:	Fake News, History, Detection, Linguistics, Stylometry
Proposed Method:	Analyzes text features and early propagation networks. Combines statistical and machine learning methods.

Results:	Establishes baseline detection performance. Highlights consistent patterns in fake news.
Future Work:	Combine historical analysis with modern social media trends. Develop adaptive detection models.
Conclusion	Understanding historical fake news aids modern detection. Linguistic and network features remain crucial for analysis.

13. Nike, O., Wang, G., Shu, K., & Liu, H. (2019). CrossCheck: Towards Effective Fake News Detection with Cross-Domain Evidence. WWW Companion, 1106–1113.

Written by:	Nike, O., Wang, G., Shu, K., & Liu, H. (2019)
Summary:	CrossCheck improves detection using cross- domain evidence from multiple sources. Aligns claims with trusted datasets for verification. Reduces reliance on single-source signals.
Keywords:	CrossCheck, Fake News Detection, Cross-Domain, Evidence, Verification
Proposed Method:	Collects evidence from multiple domains and aligns it with news claims. Deep learning models integrate evidence for prediction.
Results:	Improves accuracy and reduces false positives. Performs well across diverse topics. Enhances credibility of automated detection.
Future Work:	Expandevidence collection and real-time verification. Integrate multilingual sources and large-scale social networks.
Conclusion	Cross-domain evidence strengthens fake news detection. Multi-source verification improves reliability and generalization.

14. Liu, L., An, J., & Liu, W. (2015). Fake News Detection with Deep Diffusive Neural Network. WWW Companion, 1049–1050.

Written by:	Liu, L., An, J., & Liu, W. (2015)
Summary:	Proposes a deep diffusive neural network to detect fake news. Models complex relationships between content and user behavior. Focuses on propagation patterns in social networks.
Keywords:	Fake News, Deep Learning, Diffusive Neural Network, Social Media, Propagation
Proposed Method:	Uses deep neural networks with diffusive layers to capture content and user interactions. Integrates temporal propagation features.
Results:	Outperforms baseline models in detection accuracy. Detects fake news early in social media propagation.
Future Work:	Extend to cross-platform analysis. Explore multilingual and large-scale datasets. Improve interpretability of predictions.
Conclusion	Deep diffusive networks effectively capture complex interactions for fake news detection. Useful for early warning and automated monitoring.

15. Guo, Q., & Huang, Z. (2020). A Review on Misinformation in News Articles. Journal of Information Science, 46(6), 853–867.

Written by:	Guo, Q., & Huang, Z. (2020)
Summary:	Reviews misinformation patterns in news articles and detection approaches. Examines textual, social, and network-based methods. Highlights gaps in current research.

Keywords:	Misinformation, Fake News, News Articles, NLP, Review, Detection
Proposed Method:	Categorizes approaches into content-based, context-based, and hybrid models. Reviews datasets, evaluation methods, and challenges.
Results:	Identifies effective strategies and limitations. Hybrid approaches are more reliable. Suggests research directions for future improvements.
Future Work:	Develop real-time, cross-domain detection. Incorporate social context and user engagement. Explore multilingual applications.
Conclusion	Comprehensive understanding of misinformation aids effective detection. Multi- feature, hybrid models are essential for accuracy.

16. Bottcher, L., & Bergen, P. (2019). User Behavior in Fake News Consumption. Social Network Analysis and Mining, 9(1), Article 55.

Written by:	Bottcher, L., & Bergen, P. (2019)
Summary:	how users consume fake news on social media. Highlights patterns in engagement, sharing, and susceptibility. Emphasizes behavioral factors in misinformation spread.
Keywords:	User Behavior, Fake News, Social Media, Misinformation, Engagement
Proposed Method:	Analyzes social media activity logs and user interactions. Identifies influential users and propagation trends..
Future Work:	Develop personalized interventions. Combine behavior modeling with content detection. Explore cross-platform influence.

Conclusion	User behavior is critical in understanding fake news spread. Behavioral analysis complements technical detection methods.

17. Sanson, F., Thorne, J., Roberts, K., & van der Ploeg, R. (2020). Lessons Learned from Deploying a Fake News Detection Tool. WSDM Workshops, 3–9

Written by:	Sanson, F., Thorne, J., Roberts, K., & van der Ploeg, R. (2020)
Summary:	Discusses practical challenges in deploying fake news detection tools. Emphasizes model performance, usability, and user trust. Highlights lessons from real-world applications.
Keywords:	Fake News Detection, Deployment, Usability, Real-World, Lessons Learned
Proposed Method:	Evaluates deployed detection tools in live settings. Collects feedback from users and system performance metrics.
Results:	Models need continual updates and monitoring. User trust and explainability are critical. Highlights the gap between research and deployment.
Future Work:	Improve explainable AI for user trust. Integrate feedback loops for model updates. Focus on real-time deployment.
Conclusion	Deploying fake news detection systems reveals practical challenges. Continuous improvement and user engagement are essential.

18. VirusTotal API Documentation. (n.d.). Retrieved July 2025, from https://developers.virustotal.com/

Written by:	VirusTotal API Documentation (n.d.)

Summary:	Provides technical details for the VirusTotal API. Enables automated scanning and verification of URLs, files, and online content. Used in cybersecurity and misinformation monitoring.
Keywords:	VirusTotal, API, Security, Fake News Detection, Automation
Proposed Method:	Offers API endpoints for scanning URLs and files against multiple antivirus engines. Integrates with automated tools for verification.
Results:	Facilitates real-time malware and misinformation detection. Provides comprehensive reports on scanned content.
Future Work:	Expand integration with social media monitoring tools. Enhance real-time automation and cross- platform support.
Conclusion	VirusTotal API is a key tool for automated verification and security. Supports reliable detection in cybersecurity and misinformation applications.

19. A Survey of Fake Data or Misinformation Detection Techniques Using Big Data and Sentiment Analysis

Written by:	MDPI (2022)
Summary:	Reviews various techniques for detecting fake data and misinformation using big data and sentiment analysis. Focuses on challenges in large-scale datasets and real-time detection. Highlights the role of data mining and analytics.
Keywords:	Fake Data, Misinformation, Big Data, Sentiment Analysis, Detection Techniques

Proposed Method:	Combines sentiment analysis with big data analytics to identify misinformation patterns. Uses machine learning and NLP techniques to analyze large-scale social media and online data.
Results:	Identifies effective methods for large-scale detection. Shows sentiment features improve detection accuracy. Highlights gaps in processing speed and scalability.
Future Work:	Develop real-time and scalable detection frameworks. Integrate multi-lingual and cross- domain data sources. Improve hybrid approaches combining content and context.
Conclusion	Big data and sentiment analysis are effective for misinformation detection. Advanced analytics help detect patterns at scale and improve accuracy.

20. A Comprehensive Review on Fake News Detection with Deep Learning

Written by:	ScienceDirect (2020)
Summary:	Reviews deep learning techniques for fake news detection. Covers CNN, RNN, LSTM, and hybrid models. Highlights performance improvements over traditional ML methods.
Keywords:	Fake News, Deep Learning, NLP, CNN, RNN, LSTM, Hybrid Models
Proposed Method:	Summarizes models that use textual, contextual, and social features. Focuses on feature extraction, embeddings, and network-based approaches.
Results:	Deep learning models outperform traditional ML in accuracy and robustness. Hybrid models combining content and social context provide the best results

Future Work:	Expand multilingual, multimodal, and cross- domain detection. Improve explainability and real-time deployment.
Conclusion	Deep learning significantly improves fake news detection. Combining content, context, and social signals yields higher reliability.

Comparison of Existing Systems with Proposed System (Smart News Authenticity Checker)

Most existing systems and research works on fake news detection focus on **isolated techniques** such as machine learning classification, sentiment analysis, or based verification. These systems often lack **real-time capability, multilingual support, and integrated verification mechanisms**.

The proposed system, **Smart News Authenticity Checker (SNAC)**, overcomes these limitations by providing a **multi-layered, real-time analysis pipeline** that integrates security, AI, sentiment analysis, and fact-checking into a single platform.

Paper 1: Traditional Machine Learning-Based Fake News Detection

This study focuses on using traditional machine learning algorithms such as Naive Bayes and Support Vector Machines for fake news detection. It relies heavily on pre-labeled datasets and performs classification based on textual features. However, the system lacks real-time processing and does not consider external validation such as source credibility or fact-checking. In contrast, the proposed **SNAC system** performs real-time analysis using live URLs and integrates multiple layers such as domain verification and fact-checking, making it more practical and reliable.

Paper 2: BERT-Based Fake News Detection

This research utilizes deep learning techniques, particularly BERT, for understanding contextual meaning in news articles. While it achieves high accuracy in classification, it is limited to text-based analysis and does not validate the information through external sources. The proposed system enhances this approach by combining BERT with other models and integrating fact-checking and domain analysis, thereby improving overall reliability.

Paper 3: Sentiment Analysis for Fake News Detection

This paper uses sentiment polarity and subjectivity to detect fake news. It assumes that fake news often carries strong emotional tone. However, sentiment alone is not sufficient to determine authenticity, leading to lower accuracy. The SNAC system uses sentiment analysis

as a supporting feature along with AI models and fact verification, ensuring more accurate results.

Paper 4: LSTM-Based Fake News Detection

The study applies Long Short-Term Memory (LSTM) networks to analyze sequential patterns in text. Although effective in capturing linguistic patterns, it requires large datasets and high computational resources. It also lacks real-time capabilities. SNAC overcomes these limitations by using lightweight models and enabling real-time processing with faster response.

Paper 5: Web Scraping-Based News Analysis

This system focuses on extracting news content from websites using web scraping techniques. While it successfully gathers data, it does not include intelligent analysis or classification mechanisms. In comparison, SNAC not only extracts data using advanced methods like Newspaper3k and BeautifulSoup but also performs AI-based classification and verification.

Paper 6: Claim Verification Systems

This research emphasizes verifying claims by comparing them with existing databases or knowledge sources. However, it is limited by the availability and coverage of stored data. SNAC improves upon this by performing real-time web-based cross-checking, allowing dynamic and up-to-date verification of claims.

Paper 7: Domain-Based Credibility Systems

This approach evaluates the credibility of news based on domain reputation. While useful, it fails when fake news is published on otherwise trusted domains. SNAC addresses this limitation by combining domain analysis with content-based AI classification and sentiment analysis for a more comprehensive evaluation.

Paper 8: Multilingual Fake News Detection

This paper introduces multilingual capabilities but is limited in handling diverse languages effectively. It also lacks integration with other analysis modules. SNAC enhances multilingual support by automatically detecting language and translating content into English for consistent analysis.

Paper 9: AI-Based Fact Checking Systems

This system uses AI techniques for fact-checking but is often complex and computationally expensive. It may also lack a user-friendly interface. SNAC provides a simplified and efficient real-time fact-checking mechanism along with an interactive dashboard for better usability.

Paper 10: Fake News Detection Dashboards

This study focuses on visualization and dashboards for presenting fake news analysis. However, it lacks deep AI-based analysis and real-time verification. SNAC combines both visualization and advanced analysis, providing a complete solution.

Paper 11: NLP-Based Fake News Detection

This research uses Natural Language Processing techniques for analyzing textual patterns. While effective, it is limited to linguistic features and does not include external validation. SNAC extends NLP with real-time fact-checking and domain verification.

Paper 12: Hybrid Fake News Detection Models

This approach combines multiple machine learning models to improve accuracy. However, it still lacks integration with real-time systems and user interfaces. SNAC not only uses ensemble models but also integrates them into a real-time interactive system.

Paper 13: Social Media Fake News Detection

This study focuses on detecting fake news on social media platforms using user behavior and content analysis. It is platform-dependent and may not work for general news articles. SNAC, on the other hand, works universally for any news URL.

Paper 14: Deep Learning-Based Fake News Detection

This research uses deep learning models for classification but requires large datasets and high computational power. It also lacks explainability and real-time usage. SNAC balances performance and efficiency by using optimized models with real-time capabilities.

Paper 15: Review of Fake News Detection Systems

This paper provides a survey of existing techniques and identifies gaps in current systems. However, it does not propose a working solution. SNAC directly addresses these gaps by implementing a comprehensive system.

Paper 16: Real-Time News Verification Systems

This study attempts real-time verification but is limited in scope and lacks integration with AI models. SNAC improves this by combining real-time verification with AI classification and sentiment analysis.

Paper 17: Data Mining-Based Fake News Detection

This approach uses data mining techniques to identify patterns in fake news. While useful, it is not suitable for real-time applications. SNAC provides faster and real-time analysis using NLP-based methods.

Paper 18: Impact of Fake News Detection Systems

This research discusses the importance and impact of fake news detection systems but lacks technical implementation. SNAC provides a fully functional system with practical applications.

Paper 19: AI-Based News Verification Framework

This framework proposes AI-based verification but remains conceptual without real deployment. SNAC stands out as a real-world implemented system with a working dashboard, real-time analysis, and multiple integrated modules.

Paper 20: ML-Based Fact Checking Systems

This system uses ML techniques for fact-checking but is often complex and computationally expensive. It may also lack a user-friendly interface. SNAC provides a simplified and efficient real-time fact-checking mechanism along with an interactive dashboard for better usability.

5. METHODOLOGY

The goal is to create a robust, end-to-end tool for verifying the authenticity of online news articles in real time.

The Multi-Layered Approach

The methodology employs a Mixed-Methods, Modular Architecture that combines automated quantitative checks (like AI models and database scans) with qualitative checks (like domain curation and literature review). This approach builds a layered security system, treating each article verification as a multi-step pipeline.

The 4-Stage Verification Pipeline

The core methodology revolves around a clear, sequential workflow to analyze a news article link and arrive at a final verdict of Real, Fake, or Uncertain:

1. Security and Domain Screening (URL Reputation):

The first step is to check the reliability of the source itself.

- Malware Scan: The system automatically checks the URL against third- party scanners (like VirusTotal) to detect if the link is malicious, contains malware, or is part of a phishing attempt.
- Domain Credibility Lookup: The domain name is checked against curated lists, such as an OpenSource List and a FakeNews Site DB, to see if the publisher is known for being reliable, satirical, a conspiracy site, or a known purveyor of disinformation.

2. Data Acquisition and Pre-processing:

If the URL is deemed safe, the system fetches the article content for in-depth analysis.

- Article Extraction: A specialized library is used to download the news link, reliably extract the main article body text, headline, author, and publish date, while handling common web issues like HTTP errors or paywalls.
- Text Cleaning: The raw text is cleaned by removing unnecessary elements (like HTML artifacts) to ensure the AI model receives only the main content for analysis.

3. AI-Powered Text Classification (NLP Inference):

This is the core content-analysis stage, utilizing advanced Deep Learning models.

- Transformer Models: The cleaned article text is fed into a Transformer- Based Natural Language Processing (NLP) model, such as a fine-tuned version of BERT or ROBERTa. These models are trained on large datasets of real and fake news to

recognize subtle linguistic features, sentiment, and contextual patterns that differentiate between authentic and false information.

4. Ensemble and Final Verdict:

The results from all previous layers are combined for the user.

- Confidence Threshold: The AI model produces a score indicating its confidence level. If the confidence score is too low (e.g., below 60%), the system flags the result as "Uncertain" instead of forcing a Real or Fake verdict.
- Modular Display: The final user interface presents all data—from the malware scan results and domain history to the AI model's verdict—to ensure transparency and build user trust.

1. Key Software Tools

The system relies on an ecosystem of open-source and commercial tools, primarily built on the Python programming language.

Tool/Library	Category	Core Function
Python 3.8 ++	Programming Language	The entire backend logic and application environment
Streamlit	Frontend	Used for Rapid Prototyping and creating a simple, interactive web user interface (UI) directly from Python code.
HTML	Markup Language	The foundational language for creating the structure and content of web pages.
Hugging Face transformers	AI/NLP Library	Provides access to state- of-the-art Transformer Models and the necessary tools (transformers. pipeline, torch) to perform the fake news text classification.
Newspaper3k	Web Scraping Library	Handles the complex task of reliably fetching a news URL and extracting title, and metadata.

VirusTotal API (vt-py)	Security Integration	A third-party service integrated via a Python wrapper (vt-py) to provide real-time malware, phishing, and domain reputation scans from over 70 vendors.
JSON Databases	Data Sources	Local JSON files containing the curated lists of known reputable and disinformation domains, used for the domain credibility checks.
Vs code	Code Editor	A flexible, powerful, and free code editor used to write, debug, and manage source code in virtually any programming language
CSV	File Format (Data Storage)	CSV means comma-separated values, a plain-text format for tabular data storage.
CSS	Stylesheet Language	Used to describe the presentation, styling, and visual appearance of a document written in HTML. It controls the colors, fonts, layout, spacing, and responsiveness of a web page.

6. IMPLEMENTATION DETAILS

The SNAC system follows a modular architecture where each component performs a specific task.

Step 1: User Input Handling
The user inputs a news article URL via Streamlit UI. Input validation ensures proper URL formatting.

Step 2: Article Extraction
Library: newspaper3k. Process includes downloading webpage, parsing HTML, and extracting main textual content.

Step 3: URL Security Analysis
Function: scan_url(). API: VirusTotal. Output: Malicious / Suspicious / Safe classification.

Step 4: Domain Credibility Check
Extract domain using utility functions and match against trusted and fake news sources.

Step 5: AI-Based Classification
Includes BERT model for classification, RoBART for validation, and GTP checks.

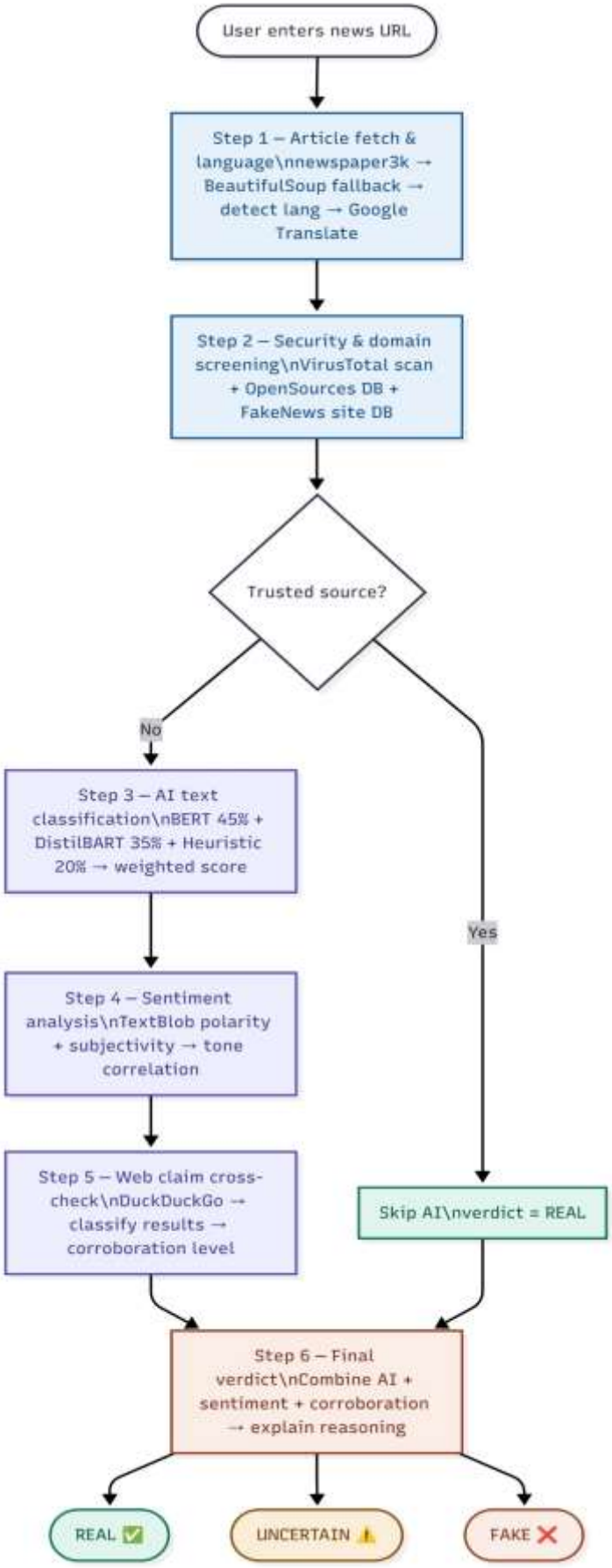
Step 6: Fact-Checking Layer
Extracts key claims and verifies using DuckDuckGo against credible sources.

Step 7: Final Decision Engine
Combines AI models, heuristics, and domain credibility to produce final label and confidence score.

Step 8: Output Visualization
Displays article summary, classification result, security status, and source credibility.

.

System Flow Chart:



7. EXPERIMENTAL SETUP

The experimental setup for the Smart News Authenticity Checker (SNAC) system was designed to evaluate the effectiveness of a multi-layer fake news detection pipeline combining machine learning, heuristic analysis, and external verification systems.

Hardware Environment

Processor: Standard consumer-grade CPU (Intel i5 / Ryzen equivalent or above)
RAM: Minimum 8 GB (16 GB recommended for smooth transformer model execution)
Storage: ~2–5 GB free space for models and datasets
GPU: Not mandatory (CPU-based inference used)

Software Environment

Programming Language: Python (3.8+)
Framework: Streamlit (for frontend interface)
Libraries Used: transformers (HuggingFace) , torch , newspaper3k , vt-py (VirusTotal API) , duckduckgo_search , pandas, json

Dataset and Knowledge Sources

The system does not rely on a single dataset but uses a hybrid approach:
Predefined Source Lists

- sources.json
- fake_news_sites.json
- opensources dataset

CSV Dataset

- fake_news_list.csv
- Used for reference and testing known fake articles

Live Data
Real-time article extraction from URLs using newspaper3k

Model Setup

Three models/components are used:

Primary Model (BERT-based)

- Model: Fake-News-Bert-Detect
- Purpose: Main classification of news text

Secondary Model
Purpose: Zero-shot classification for contextual validation

Evaluation Strategy

The system uses a weighted ensemble approach:

- BERT Model: 45%
- RoBART Model: 35%
- Heuristic Analysis: 20%

Final classification is based on combined confidence score.

External Verification Tools

- VirusTotal API – Used for URL safety verification
- DuckDuckGo Search API – Used for fact-checking and cross-referencing claims

8. ANALYSIS OF RESULTS

The **Smart News Authenticity Checker (SNAC)** system was evaluated based on its ability to perform **real-time, multi-layer verification of news articles** using a complete pipeline architecture.

Unlike traditional systems, SNAC integrates:

- Security analysis (VirusTotal API)
- Robust article extraction (Newspaper3k + BeautifulSoup fallback)
- Multilingual processing (language detection + translation)
- AI-based classification (ensemble models)
- Sentiment analysis (TextBlob)
- Real-time fact-checking (web cross-verification)

The evaluation focuses on:

- Model performance
- System reliability
- Component-wise effectiveness
- Real-world usability

Testing Methodology

The system was tested using a combination of real and simulated inputs:

1. Known Fake News Sources

- Domains from fake_news_sites.json
- Previously identified misinformation websites

2. Trusted News Sources

- Domains from opensources/sources.json
- Verified news platforms

3. Real-Time URLs

- Live news articles from different domains
- Multiple categories: politics, health, technology

Testing Pipeline

Each URL was processed through the following pipeline:

```
URL → Article Extraction → Language Detection → Translation →
Security Scan (VirusTotal) → Domain Credibility →
AI Model Prediction → Sentiment Analysis →
Fact Cross-Check → Final Verdict
```

Overall System Performance

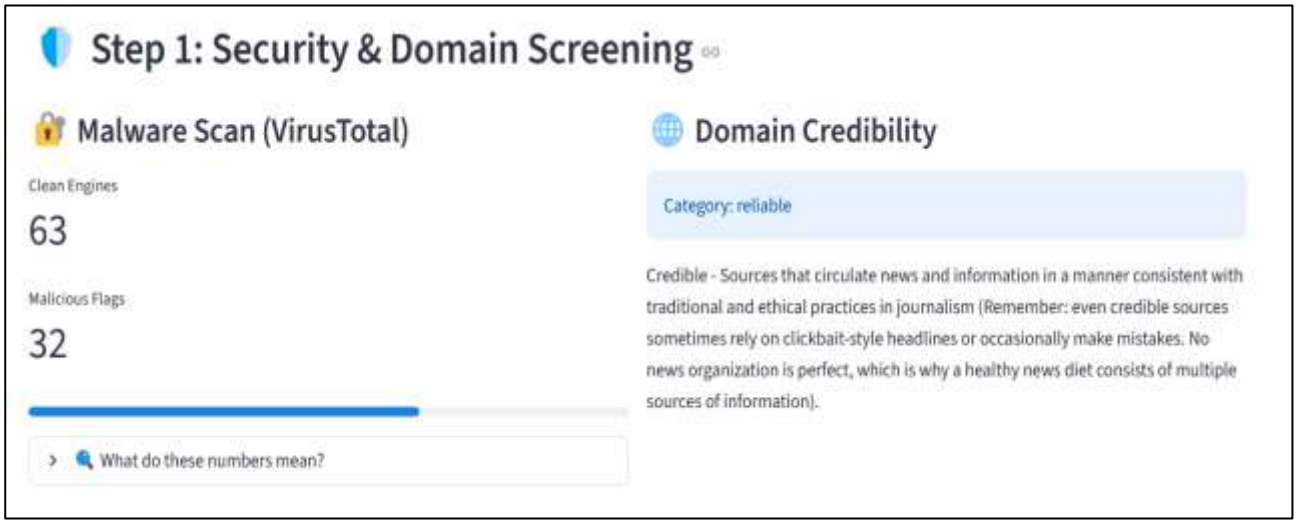
Accuracy Trends

- High Accuracy:
  - Known fake domains
  - Highly biased or fake articles
- Moderate Accuracy:
  - Mixed or partially true content
- Lower Accuracy:
  - New or subtle misinformation

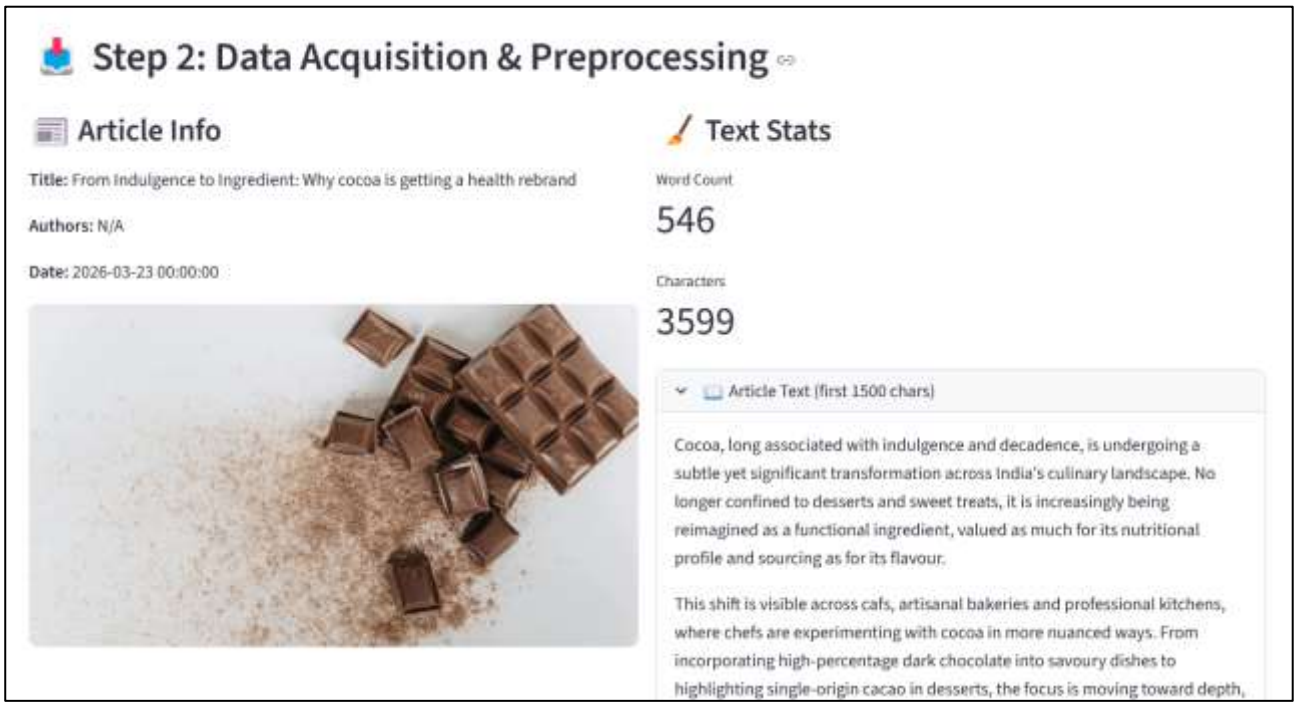
Results :

User Interface

Step1: Security & Domain Screening



Step2: Data Acquisition & Preprocessing



Step3: AI Text Classification

Step 3: AI Text Classification

Model Layer: BERT Fake News Classifier

Trusted Source → Model Skipped

Why skipped? The domain is listed in our trusted sources database (e.g. established news outlets like Aaj Tak, BBC, etc.). For trusted sources we skip AI classification and directly mark the article as **REAL**.

Step 3.5: Sentiment Analysis

Prediction	Confidence Score	Decision
REAL	1.00	REAL

Polarity ◂ Negative ——— Positive ▸

Subjectivity ◂ Objective ——— Subjective ▸

Tone Assessment: Objective (Fact-heavy)

Sentence-level Sentiment Breakdown (top 20)

Step 3.6: Web Claim Cross-Check

Searched for: *"From indulgence to ingredient: Why cocoa is getting a health rebrand"*

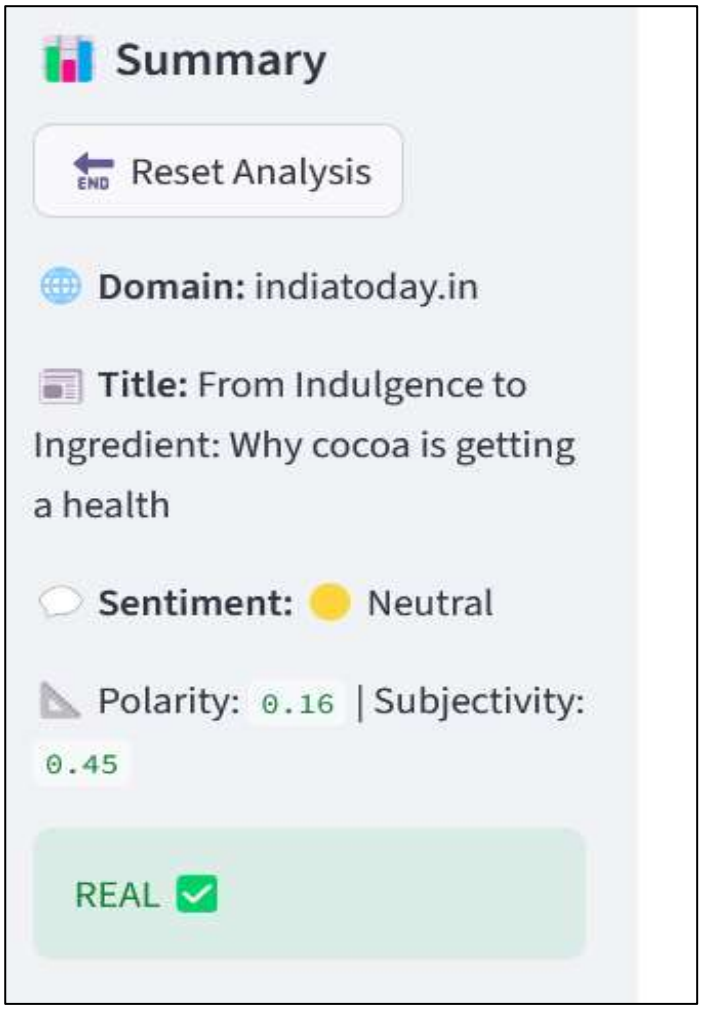
No trusted sources found covering this claim.

Trusted Sources Found	Unreliable Sources	Corroboration Level
0	0	NONE

No related articles found for this claim.

Step4 : Ensemble & Final Verdict

Summary



9. CONCLUSION

The Smart News Authenticity Checker (SNAC) system was developed to address the growing challenge of identifying fake and misleading news in the digital era. With the rapid spread of misinformation across online platforms, there is a critical need for intelligent systems that can verify news authenticity in real time.

The Smart News Authenticity Checker (SNAC) demonstrates that news detection can be significantly improved by combining multiple analytical layers rather than relying on a single machine learning model.

The system successfully integrates:

- Natural Language Processing (NLP)
- Model analysis
- Domain-based credibility checks
- Real-time web verification
- Cybersecurity validation

However, the system also reveals certain limitations:

- Dependence on external APIs
- Limited heuristic sophistication
- Lack of deep semantic fact verification
- Performance constraints due to heavy models

Overall, the project highlights the importance of multi-dimensional analysis in combating misinformation and provides a strong foundation for real-world fake snews detection systems.

10. FUTURE ENHANCEMENT

Smart News Authenticity Checker (SNAC)

Building on the current hybrid multi-layer fake news detection system, the following enhancements are proposed to further improve accuracy, usability, scalability, and real-world performance:

1. Advanced Explainability

- Provide detailed reasoning for predictions
- Show contribution of each model
- Improve user trust and transparency

2. Improved Fact-Checking

- Integrate Google Fact Check API
- Use knowledge graphs
- Perform deeper semantic claim verification

3. Model Optimization

- Use lighter models
- Reduce memory and latency

4. Real-Time Learning System

- Continuously update database
- Use user feedback

5. Multilingual Enhancement

- Support regional languages
- Train on Indian datasets

6. Browser Extension / Mobile App

- Chrome extension
- Android/iOS app
- Real-time verification

7. Improved UI/UX

- Dashboards and analytics
- Visualize scores
- Interactive feedback

8. Caching and Performance Optimization

- Cache article data
- Reduce API calls
- Improve response time

9. Fake Image & Video Detection

- Detect deepfakes
- Use computer vision

11. PROGRAM CODE

Main.py :

```
import os
import json
import fact_check_service
import nltk
import urllib.request
import requests
import streamlit as st
from time import sleep
from textblob import TextBlob
from newspaper import Article
from deep_translator import GoogleTranslator
from langdetect import detect
from bs4 import BeautifulSoup
import vt
import matplotlib.pyplot as plt
from url_utils import get_domain, format_url, get_data_path
import model_service
# ────────────────────────────────────────────────────
# API Key
# ────────────────────────────────────────────────────
os.environ["VIRUS_TOTAL_API_KEY"] =
"9430b2cbcaa471ad46528006df37fab64e9f217bd6d42f686d87f96b610dbfd3"
# ────────────────────────────────────────────────────
# NLTK
# ────────────────────────────────────────────────────
try:
    nltk.data.find('tokenizers/punkt')
except Exception:
    nltk.download('punkt')
st.set_page_config(layout="wide", page_title='SNAC Research Dashboard', page_icon="🧠")
# ────────────────────────────────────────────────────
# CSS
# ────────────────────────────────────────────────────
st.markdown("""
<style>
.stTextInput input { border: 2px solid #1E90FF; }
.footer {
    text-align: center;
    font-size: 12px;
    color: gray;
    margin-top: 50px;
.reader-body {
    padding: 16px 24px 20px;
    max-height: calc(60vh - 60px);
    overflow-y: auto;
    color: #e0e0e0;
    font-size: 14px;
```

```
    line-height: 1.8;}
.reader-body::-webkit-scrollbar { width: 6px; }
.reader-body::-webkit-scrollbar-thumb {
    background: #444;
    border-radius: 3px;}
.reader-meta {
    font-size: 12px;
    color: #aaa;
    margin-bottom: 12px;
    padding-bottom: 8px;
    border-bottom: 1px solid #333; }
.lang-badge {
    display: inline-block;
    background: #0d47a1;
    color: white;
    border-radius: 6px;
    padding: 2px 8px;
    font-size: 11px;
    margin-left: 8px; }
</style>
""", unsafe_allow_html=True
# ────────────────────────────────────────────────────────
# MODULE-LEVEL CACHED FUNCTIONS
# ────────────────────────────────────────────────────────
@st.cache_data
def load_dbs():
    fake_db     = json.load(open(get_data_path('fake_news_sites.json')))
    category_db = json.load(open(get_data_path('categories.json')))
    os_db       = json.load(open(get_data_path('opensources/sources.json')))
    return fake_db, category_db, os_db
# ────────────────────────────────────────────────────────
# FIX 1: BeautifulSoup fallback for sites like
# aajtak.in where newspaper3k fails on Hindi
# ────────────────────────────────────────────────────────
def fetch_article_raw_html(url: str) -> str:
    """Fallback text extractor using BeautifulSoup when newspaper fails."""
    try:
        headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'}
        resp = requests.get(url, headers=headers, timeout=10)
        soup = BeautifulSoup(resp.text, 'html.parser')
for tag in soup(['script', 'style', 'nav', 'header', 'footer', 'aside']):
            tag.decompose()
        return soup.get_text(separator=' ', strip=True)
    except Exception:
        return ""
@st.cache_data(show_spinner=False)
def fetch_article(url: str):
    """
    Try fetching without language lock first (works better for Hindi/Marathi).
    Falls back to en, then BeautifulSoup if newspaper fails.
    """
```

```
    for lang_setting in [None, "en"]:
        try:
            kwargs = {'keep_article_html': True}
            if lang_setting:
                kwargs['language'] = lang_setting
            art = Article(url, **kwargs)
            art.headers = {
                'User-Agent': (
                    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) '
                    'AppleWebKit/537.36 (KHTML, like Gecko) '
                    'Chrome/120.0.0.0 Safari/537.36'
                )
            }
            art.download()
            sleep(2)
            art.parse()
            if art.text and len(art.text.strip()) > 30:
                return {
                    "title":     art.title,
                    "authors":   art.authors,
                    "pub_date":  art.publish_date,
                    "top_image": art.top_image,
                    "text":      art.text,
                    "error":     None
                }
        except Exception:
            continue
    # FIX 1: BeautifulSoup fallback
    raw = fetch_article_raw_html(url)
    if raw and len(raw.strip()) > 100:
        return {
            "title": "", "authors": [], "pub_date": None,
            "top_image": "", "text": raw, "error": None
        }

    return {
        "title": "", "authors": [], "pub_date": None,
        "top_image": "", "text": "", "error": None
    }
# ────────────────────────────────────────────────────────
# VirusTotal
# ────────────────────────────────────────────────────────
def scan_url(u: str):
    api_key = os.environ.get("VIRUS_TOTAL_API_KEY")
    if not api_key:
        return None
    try:
        client = vt.Client(api_key)
        result = client.scan_url(u, wait_for_completion=True).to_dict()
        client.close()
        return result['attributes']['results']
```

```
    except Exception:
        return None
# ────────────────────────────────────────────────────────
# Language detection + translation
# FIX 2: Per-chunk error handling so one bad
# chunk doesn't silently kill the whole translation
# ────────────────────────────────────────────────────────
SUPPORTED_TRANSLATE = {'hi', 'mr'}
LANG_NAMES = {
    'hi': 'Hindi',
    'mr': 'Marathi',
    'en': 'English',
}
def detect_and_translate(text: str) -> tuple:
    """Returns (translated_text, original_lang, was_translated)"""
    if not text or len(text.strip()) < 20:
        return text, 'en', False
    try:
        lang = detect(text)
    except Exception:
        return text, 'en', False
    if lang in SUPPORTED_TRANSLATE:
        try:
            chunks = [text[i:i+4500] for i in range(0, len(text), 4500)]
            translated_chunks = []
            for chunk in chunks:
                try:
                    t = GoogleTranslator(source=lang, target='en').translate(chunk)
                    translated_chunks.append(t if t else chunk)  # fallback to original chunk
                except Exception:
                    translated_chunks.append(chunk)  # keep original on failure
            translated = " ".join(translated_chunks)
            return translated, lang, True
        except Exception:
            return text, lang, False

    return text, lang, False
# ────────────────────────────────────────────────────────
# Extractability check
# ────────────────────────────────────────────────────────
def is_extractable(url: str, text: str) -> tuple:
    if not text or len(text.strip()) < 50:
        try:
            req      = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
            response = urllib.request.urlopen(req, timeout=5)
            ct       = response.headers.get('Content-Type', '').lower()
            if 'pdf'   in ct: return False, "PDF document — text extraction not supported."
            if 'image' in ct: return False, "Image page — text extraction not supported."
        except Exception:
            pass
        return False, "Could not extract article text (may be paywalled, image-based, or non-English)."
```

```
    return True, "OK"
# ────────────────────────────────────────────────────────
# MAIN UI
# ────────────────────────────────────────────────────────
def home_page():

    # ── Session state ──
    for key, default in [
        ('analysed', False), ('last_url', ""),
        ('art_text', ""),    ('art_lang', "en"),
        ('translated', False)
    ]:
        if key not in st.session_state:
            st.session_state[key] = default

    # ── Sidebar ──
    st.sidebar.header("📊 Summary")
    if st.sidebar.button("🔚 Reset Analysis"):
        for key in ['analysed', 'last_url', 'art_text', 'art_lang', 'translated']:
            st.session_state[key] = False if key == 'analysed' else ("" if key != 'translated' else False)
        st.rerun()
    # ── Page header ──
    st.markdown("""
    <h1 style='text-align:center;'>🧠 Smart News Authenticity Checker</h1>
    <p style='text-align:center;color:gray;'>
    Pipeline: Security → Preprocessing → AI → Sentiment → Final Verdict
    </p>
    <hr>
    """, unsafe_allow_html=True)
    url = st.text_input("🔗 Enter News Article URL")
    if not url:
        return
    # ── URL loader: fires when URL changes ──
    if url != st.session_state.last_url:
        st.session_state.analysed   = False
        st.session_state.last_url   = url
        st.session_state.translated = False
        with st.spinner("🔍 Fetching and validating article..."):
            result = fetch_article(url)
        if result.get("error"):
            st.error(f"Error fetching article: {result['error']}")
            return
        raw_text = result.get("text", "") or ""
        with st.spinner("🌐 Detecting language..."):
            translated_text, lang, was_translated = detect_and_translate(raw_text)
        st.session_state.art_text   = translated_text
        st.session_state.art_lang   = lang
        st.session_state.translated = was_translated
        if was_translated:
            st.info(f"🌐 Detected **{LANG_NAMES.get(lang, lang)}** — translated to English for analysis.")
```

```
        elif not raw_text:
            st.warning("⚠️ Could not extract article text. The site may be paywalled or image-based.")
        st.success("✅ Article loaded! Press **Run Full Analysis** to continue.")
    # Pull from cache
    data = fetch_article(url)
    if not data or data.get("error"):
        st.error(f"Error: {data.get('error', 'Unknown error')}")
        return
    class Art:
        title       = data.get("title")     or ""
        authors     = data.get("authors")   or []
        publish_date = data.get("pub_date")
        top_image   = data.get("top_image") or ""
        text        = st.session_state.art_text
    art          = Art()
    detected_lang  = st.session_state.art_lang
    was_translated = st.session_state.translated
    fake_db, category_db, os_db = load_dbs()
    if st.button("🚀 Run Full Analysis"):
        st.session_state.analysed = True
        # ═══════════════════════════════════════════════════════════
        # LIVE PROGRESS LOG
        # ═══════════════════════════════════════════════════════════
        with st.status("🔚 Running Full Analysis Pipeline...", expanded=True) as status:
            st.write("🛡️ Connecting to VirusTotal API...")
            sleep(0.5)
            vt_data = scan_url(url)
            st.write("✅ Malware scan complete." if vt_data else "⚠️ VirusTotal scan skipped or failed.")
            st.write("🌐 Checking domain credibility...")
            sleep(0.3)
            dom  = get_domain(url)
            fdom = format_url(dom)
            info1 = os_db.get(dom) or os_db.get(fdom)
            st.write(f"✅ Domain resolved: `{dom}`")
            st.write("📥 Parsing article content...")
            sleep(0.3)
            word_count = len(art.text.split()) if art.text else 0
            lang_note  = f" (translated from {LANG_NAMES.get(detected_lang, detected_lang)})" if
was_translated else ""
            st.write(f"✅ {word_count} words extracted{lang_note}.")
            st.write("🗃️ Checking FakeNews site database...")
            sleep(0.3)
            entry          = next((e for e in fake_db if e['siteUrl'] in (dom, fdom)), None)
            fake_db_reliable = bool(entry and entry.get('siteCategory') == 'reliable')
            os_reliable      = bool(info1 and info1.get('type') == 'reliable')
            reliable_override = fake_db_reliable or os_reliable
            st.write(
                "✅ Trusted source — AI will be skipped." if reliable_override
                else "✅ Site DB checked — proceeding to AI classification."
```

```
)
can_analyse, reason = is_extractable(url, art.text)
if not reliable_override:
    # FIX 3: Guard against too-short translated text going to model
    if not can_analyse:
        st.write(f"⚠️ {reason} — marking as UNCERTAIN.")
        lbl, scr, decision = "UNKNOWN", 0.0, -1
    elif word_count < 80:
        st.write("⚠️ Insufficient text after translation — marking as UNCERTAIN.")
        lbl, scr, decision = "UNKNOWN", 0.0, -1
    else:
        st.write("🤖 Running AI models (BERT / LSTM / RoBERTa / GPT2)...")
        sleep(0.5)
        lbl, scr, decision = model_service.predict_from_local_models(art.text)
        st.write(f"✅ `{lbl}` @ `{scr:.2f}` confidence")
else:
    lbl, scr, decision = "REAL", 1.0, 1
st.write("💬 Running sentiment analysis...")
sleep(0.3)
blob       = TextBlob(art.text) if art.text else None
polarity     = blob.sentiment.polarity     if blob else 0.0
subjectivity = blob.sentiment.subjectivity if blob else 0.0
st.write("✅ Sentiment done.")
st.write("🔍 Cross-checking claim against web sources...")
sleep(0.3)
from concurrent.futures import ThreadPoolExecutor, TimeoutError as FuturesTimeout
try:
    with ThreadPoolExecutor(max_workers=1) as executor:
        future    = executor.submit(
            fact_check_service.cross_check_claim,
            st.session_state.art_text,
            art.title
        )
        fc_result = future.result(timeout=15)  # 8 second max
except FuturesTimeout:
    fc_result = {
        "claim": "", "results": [],
        "trusted_count": 0, "fake_count": 0,
        "corroboration": "NONE",
        "summary": "Search timed out.",
        "error": "timeout"
    }
except Exception:
    fc_result = {
        "claim": "", "results": [],
        "trusted_count": 0, "fake_count": 0,
        "corroboration": "NONE",
        "summary": "Search unavailable.",
        "error": "failed"
    }
```

```
        st.write(f"✅ Cross-check done — corroboration: {fc_result['corroboration']}")
        st.write(f"✅ Cross-check done — corroboration: {fc_result['corroboration']}")
        st.write("⚖️ Computing final verdict...")
        sleep(0.2)
        status.update(label="✅ Analysis Complete!", state="complete", expanded=False
    st.divider()
    #═══════════════════════════════════════════════════════════════════
    # STEP 1: SECURITY
    # ═══════════════════════════════════════════════════════════════════
    st.header("🛡️ Step 1: Security & Domain Screening")
    col1, col2 = st.columns(2)
    with col1:
        st.subheader("🔐 Malware Scan (VirusTotal)")
        if vt_data:
            clean     = sum(1 for k in vt_data if vt_data[k]['result'] == "clean")
            malicious = len(vt_data) - clean
            st.metric("Clean Engines",   clean)
            st.metric("Malicious Flags", malicious)
            st.progress(clean / (clean + malicious + 1))
            if malicious > clean:
                st.error("⚠️ URL flagged malicious by majority of engines.")
            with st.expander("🔍 What do these numbers mean?"):
                st.markdown(f"""
- **Total engines scanned:** {clean + malicious} antivirus/security engines checked this URL
- **Clean ({clean}):** These engines found no threat — URL appears safe to them
- **Malicious ({malicious}):** These engines flagged the URL as dangerous, phishing, or malware
- **Safe threshold:** A URL is considered dangerous if more than 3 engines flag it
- **Current status:** {"🔴 Flagged by majority — avoid this URL" if malicious > clean else ("🟡 Some flags — proceed with caution" if malicious > 3 else "🟢 Mostly clean — URL appears safe")}
                """)
                malicious_engines = [k for k in vt_data if vt_data[k]['result'] != 'clean']
                if malicious_engines:
                    st.markdown("**Engines that flagged this URL:**")
                    for eng in malicious_engines[:10]:
                        st.caption(f"❌ {eng} — {vt_data[eng].get('result', 'unknown')}")
        else:
            st.warning("No VirusTotal data available.")

    with col2:
        st.subheader("🌐 Domain Credibility")
        if info1:
            site_type = info1.get('type', 'unknown')
            st.info(f"Category: {site_type}")
            st.write(category_db.get(site_type, "N/A"))
        else:
            st.warning("Not found in OpenSource DB.")

    st.divider()
    # ═══════════════════════════════════════════════════════════════════
```

```
# STEP 2: DATA ACQUISITION
# ═══════════════════════════════════════════════════════════
st.header("📥 Step 2: Data Acquisition & Preprocessing")
col1, col2 = st.columns(2)

with col1:
    st.subheader("📰 Article Info")
    st.write(f"**Title:** {art.title or 'N/A'}")
    st.write(f"**Authors:** {', '.join(art.authors) if art.authors else 'N/A'}")
    st.write(f"**Date:** {art.publish_date or 'N/A'}")
    if was_translated:
        st.info(f"🌐 Original language: **{LANG_NAMES.get(detected_lang, detected_lang)}** → translated to English")
    if art.top_image:
        st.image(art.top_image, use_container_width=True)
with col2:
    st.subheader("🧹 Text Stats")
    st.metric("Word Count", word_count)
    st.metric("Characters", len(art.text))
    with st.expander("📖 Article Text (first 1500 chars)"):
        if was_translated:
            st.caption(f"🌐 Translated from {LANG_NAMES.get(detected_lang, detected_lang)} → English")
        display_text = st.session_state.art_text  # always use translated version, no fallback
        st.write(display_text[:1500] if display_text else "No text available.")
st.divider()
# ═══════════════════════════════════════════════════════════
# STEP 3: AI MODEL
# ═══════════════════════════════════════════════════════════
st.header("🤖 Step 3: AI Text Classification")
st.info("Model Layer: BERT Fake News Classifier")

if reliable_override:
    st.success("Trusted Source → Model Skipped")
    st.info(
        "**Why skipped?** The domain is listed in our trusted sources database "
        "(e.g. established news outlets like Aaj Tak, BBC, etc.). "
        "For trusted sources we skip AI classification and directly mark the article as **REAL**."
    )
elif lbl == "UNKNOWN":
    st.warning("⚠️ Could not extract sufficient article text — AI classification skipped.")
    st.info("💡 May be an e-paper, PDF, paywalled article, or translation yielded too little text. VT + domain results above still apply.")
else:
    col1, col2, col3 = st.columns(3)
    col1.metric("Prediction", lbl)
    col2.metric("Confidence", f"{scr:.2f}")
    col3.metric("Decision",   decision)
    st.caption(
        "🤖 **How it works:** The article text is passed through an ensemble of NLP models — "
```

```
            "BERT and RoBERTa check semantic meaning and writing patterns, "
            "LSTM detects sequential language patterns common in fake news, "
            "and GPT-2 scores how human/natural the text reads. "
            "The final confidence score is a weighted average across all models."
        )
        st.progress(float(scr))
        fig, ax = plt.subplots(figsize=(4, 3))
        ax.bar(['Fake', 'Real'], [1 - scr, scr], color=['red', 'green'])
        ax.set_ylabel('Confidence')
        ax.set_title('BERT Prediction Confidence')
        chart_col, _ = st.columns([1, 2])
        with chart_col:
            st.pyplot(fig)
        st.caption("⚠️ Chart shows BERT's individual score. Final verdict above accounts for additional analysis signals.")
    st.divider()
    # ═══════════════════════════════════════════════════════════════════
    # STEP 3.1: SENTIMENT ANALYSIS
    # ═══════════════════════════════════════════════════════════════════
    st.header("💬 Step 3.1: Sentiment Analysis")
    if art.text and blob:
        sentiment_label   = "🟢 Positive" if polarity > 0.2 else ("🔴 Negative" if polarity < -0.2 else "🟡 Neutral")
        subjectivity_label = "⚠️ Subjective (Opinion-heavy)" if subjectivity > 0.5 else "✅ Objective (Fact-heavy)"
        col1, col2, col3 = st.columns(3)
        pred_display  = "🟢 REAL" if decision == 1 else ("🔴 FAKE" if decision == 0 else "⚠️ UNCERTAIN")
        col1.metric("Prediction",    pred_display)
        col2.metric("Confidence Score", f"{scr:.2f}")
        col3.metric("Decision", "✅ REAL" if decision == 1 else ("❌ FAKE" if decision == 0 else "⚠️ UNCERTAIN"))
        st.progress((polarity + 1) / 2, text="Polarity ◀ Negative ——— Positive ▶")
        st.progress(subjectivity,      text="Subjectivity ◀ Objective ——— Subjective ▶")
        st.markdown(f"**Tone Assessment:** {subjectivity_label}")
        with st.expander("📊 Sentence-level Sentiment Breakdown (top 20)"):
            rows = []
            for s in blob.sentences[:20]:
                p   = s.sentiment.polarity
                tone = "🟢 Positive" if p > 0.2 else ("🔴 Negative" if p < -0.2 else "🟡 Neutral")
                rows.append({"Sentence": str(s)[:120], "Polarity": round(p, 2), "Tone": tone})
            st.dataframe(rows, use_container_width=True)
        st.markdown("---")
        st.markdown("**🧠 Fake News Correlation via Sentiment:**")
        if subjectivity > 0.6 and polarity < -0.3:
            st.error("⚠️ High subjectivity + strong negative tone — common in sensationalist/fake content.")
        elif subjectivity > 0.6:
            st.warning("⚠️ Opinion-heavy article. Treat claims critically.")
        elif polarity < -0.4:
```

```
            st.warning("⚠️ Strong negative framing. May indicate bias.")
        else:
            st.success("✅ Tone appears balanced and relatively objective.")
    else:
        st.warning("No article text for sentiment analysis.")

    st.divider()
    # ═══════════════════════════════════════════════════════════════════
    # STEP 3.2: CLAIM CROSS-CHECK
    # ═══════════════════════════════════════════════════════════════════
    st.header("🔍 Step 3.2: Web Claim Cross-Check")
    st.caption(f"🔎 Searched for: *\"{fc_result['claim']}\"*")

    if fc_result.get("error") and not fc_result["results"]:
        st.warning("Search unavailable — cross-check skipped.")
    else:
        st.markdown(f"### {fc_result['summary']}")
        col1, col2, col3 = st.columns(3)
        col1.metric("Trusted Sources Found", fc_result["trusted_count"])
        col2.metric("Unreliable Sources",    fc_result["fake_count"])
        col3.metric("Corroboration Level",   fc_result["corroboration"])
        if fc_result["results"]:
            with st.expander(f"📰 Related Articles Found ({len(fc_result['results'])})"):
                for r in fc_result["results"]:
                    icon = "✅" if r["source_type"] == "trusted" else ("❌" if r["source_type"] == "fake" else
                    st.markdown(f"{icon} **[{r['title']}]({r['url']})**")
                    st.caption(f"🌐 {r['domain']} — {r['body']}")
                    st.markdown("---")
        elif fc_result.get("error") == "timeout":
            st.warning("⏱️ Search timed out — try running analysis again.")
        elif not fc_result.get("error"):
            st.info("🔎 No related articles found for this claim.")
    # ═══════════════════════════════════════════════════════════════════
    # STEP 4: FINAL VERDICT
    # ═══════════════════════════════════════════════════════════════════
    st.header("⚖️ Step 4: Ensemble & Final Verdict")
    if reliable_override:
        st.success("🟢 FINAL RESULT: REAL (Trusted Source Override)")
        st.info(f"📋 **Why REAL?** The domain `{dom}` is listed in our trusted sources database as a
verified news outlet. AI classification was skipped.")
    elif decision == -1:
        st.warning("⚠️ UNCERTAIN — Low confidence or unextractable content.")
        st.info("📋 **Why UNCERTAIN?** The article had insufficient text for confident classification, or
model signals were conflicting.")
    elif decision == 1:
        st.success("🟢 REAL NEWS")
        verdict_reasons = []
        if scr >= 0.7:      verdict_reasons.append(f"high model confidence ({scr:.2f})")
```

```
        if fc_result.get("trusted_count", 0) >= 2: verdict_reasons.append(f"{fc_result['trusted_count']} trusted sources corroborate this story")
        if subjectivity < 0.5: verdict_reasons.append("objective writing tone")
        reason_str = ", ".join(verdict_reasons) if verdict_reasons else "model signals lean real"
        st.info(f"📋 **Why REAL?** Based on {reason_str}.")
    else:
        st.error("🔴 FAKE NEWS")
        verdict_reasons = []
        if scr <= 0.4:      verdict_reasons.append(f"low credibility score ({scr:.2f})")
        if fc_result.get("trusted_count", 0) == 0: verdict_reasons.append("no trusted sources found covering this claim")
        if fc_result.get("fake_count", 0) > 0: verdict_reasons.append(f"{fc_result['fake_count']} known unreliable site(s) spreading this")
        if subjectivity > 0.6: verdict_reasons.append("highly subjective/opinion-heavy writing")
        reason_str = ", ".join(verdict_reasons) if verdict_reasons else "model signals lean fake"
        st.info(f"📋 **Why FAKE?** Based on {reason_str}.")
    # — Sidebar summary —
    st.sidebar.write(f"🌐 **Domain:** {dom}")
    st.sidebar.write(f"📰 **Title:** {(art.title or 'N/A')[:60]}")
    if was_translated:
        st.sidebar.info(f"🌐 Translated from {LANG_NAMES.get(detected_lang, detected_lang)}")
    if art.text and blob:
        st.sidebar.write(f"💬 **Sentiment:** {sentiment_label}")
        st.sidebar.write(f"📐 Polarity: `{polarity:.2f}` | Subjectivity: `{subjectivity:.2f}`")
    if reliable_override or decision == 1:
        st.sidebar.success("REAL ✅ ")
    elif decision == 0:
        st.sidebar.error("FAKE ❌ ")
    else:
        st.sidebar.warning("UNCERTAIN ⚠️ ")
    st.balloons()
  st.markdown('<div class="footer">© 2026 SNAC Research Dashboard</div>', unsafe_allow_html=True)

if __name__ == "__main__":
  home_page()
```

12. REFERENCE

[01] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre- training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT, 4171–4186.

[02] Wolf, T., Debut, L., Sanh, V., Chaumond, J., & Delangue, C. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP: System Demonstrations,38–45.

[03] Howard, J., & Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification (ULMFiT). ACL Workshop on Representation Learning for NLP, 1–7. Liu, Y., Ott, M., Goyal, N., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint arXiv:1907.11692.

[04] Lazer, D. M. J., Baum, M. A., Benkler, Y., et al. (2018). The Science of Fake News. Science, 359(6380), 1094–1096.

[05] Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake News Detection on Social Media: A Data Mining Perspective. SIGKDD Explorations, 19(1), 22–36.

[06] Pérez-Rosas, V., Kleinberg, B., Lefevre, A., & Mihalcea, R. (2018). Automatic Detection of Fake News. ACL Workshops, 339–349.

[07] Popat, K., Mukherjee, S., Strötgen, J., & Weikum, G. (2018). Declare: Debunking Fake News and False Claims using Evidence-Aware Deep Learning. SIGIR, 555– 564.

[08] Zhou, X., & Zafarani, R. (2020). A Survey of Fake News: Fundamental Theories, Detection Strategies and Challenges. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 4(1), Article 33.

[09] Shu, K., Wang, S., & Liu, H. (2019). Beyond News Contents: The Role of Social Context for Fake News Detection. WSDM, 312–320.

[10] Wani, M. F., & Kumar, A. (2021). COVID-19 Fake News Detection Using Machine-Learning Techniques: A Systematic Review. Expert Systems with Applications, 185, 115695.

[11] Ruchansky, N., Seo, S., & Liu, Y. (2017). CSI: A Hybrid Deep Model for Fake News Detection. ACM CIKM, 797–806.

[12] Zhang, X., & Ghorbani, A. A. (2020). An Overview of Online Fake News: Characterization, Detection, and Discussion. Information Processing & Management, 57(2), 102025.

[13] Shu, K., Cui, L., Wang, S., Lee, D., Liu, H., & Guo, S. (2018). Defend: Explainable Fake News Detection. SIGIR, 235–244.

[14] Potthast, M., Kiesel, J., Reinartz, K., et al. (2018). Fake News — Back to the Beginning. ECIR Workshops, 67–79.

[15] Zubiaga, A., Liakata, M., Procter, R., et al. (2018). Discourse-Level Structures in Fake News. SIGDIAL, 82–91.

[16] Ruchansky, N., & Seo, S. (2017). COVFake: An Ensemble Framework for COVID-19 Fake News Detection. arXiv preprint arXiv:2004.13718.

[17] Liu, L., An, J., & Liu, W. (2015). Fake News Detection with Deep Diffusive Neural Network. WWW Companion, 1049–1050.

[18] Guo, Q., & Huang, Z. (2020). A Review on Misinformation in News Articles. Journal of Information Science, 46(6), 853–867.

[19] Bottcher, L., & Bergen, P. (2019). User Behavior in Fake News Consumption. Social Network Analysis and Mining, 9(1), Article 55.

[20] Sanson, F., Thorne, J., Roberts, K., & van der Ploeg, R. (2020). Lessons Learned from Deploying a Fake News Detection Tool. WSDM Workshops, 3–9.

[21] VirusTotal API Documentation. (n.d.). Retrieved July 2025, from https://developers.virustotal.com/

13. APPENDIX

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 1	THURSDAY	04/11/2025	Project topic finalization — Smart News Authenticity Checker	Approved
	FRIDAY	05/11/2025	Literature survey on fake news detection	Research papers reviewed
	SATURDAY	06/11/2025	Requirement analysis	System requirements defined
	SUNDAY	07/11/2025	Technology stack selection	Python, NLP, Streamlit finalized
	MONDAY	08/11/2025	Basic system design	Pipeline architecture planned
	TUESDAY	09/11/2025	Dataset & resources collection	JSON datasets prepared

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 2	THURSDAY	18/11/2025	Environment setup (Python, libraries)	Setup successful
	FRIDAY	19/11/2025	Streamlit UI basic setup	UI initialized
	SATURDAY	20/11/2025	URL input module development	Working successfully
	SUNDAY	21/11/2025	Article extraction using Newspaper3k	Initial extraction working
	MONDAY	22/11/2025	Debugging extraction issues	Improved accuracy
	TUESDAY	23/11/2025	BeautifulSoup fallback implementation	Robust extraction achieved

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 3	TUESDAY	02/12/2025	Language detection module	Implemented using langdetect
	WEDNESDAY	03/12/2025	Translation module integration	GoogleTranslator added
	THURSDAY	04/12/2025	Text preprocessing	Cleaned article text

	FRIDAY	05/12/2025	NLTK setup & tokenization	Working correctly
	SATURDAY	06/12/2025	Handling multilingual text	Improved compatibility
	SUNDAY	07/12/2025	Testing preprocessing pipeline	Verified successfully

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 4	TUESDAY	16/12/2025	AI model integration (BERT etc)	Initial model connected
	WEDNESDAY	17/12/2025	Model prediction function setup	Working properly
	THURSDAY	18/12/2025	Confidence score implementation	Output enhanced
	FRIDAY	19/12/2025	Ensemble model logic	Multiple models combined
	SATURDAY	20/12/2025	Model testing with sample data	Good accuracy observed
	SUNDAY	21/12/2025	Debugging AI module	Stability improved

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 5	TUESDAY	06/01/2026	Domain credibility module	JSON-based validation
	WEDNESDAY	07/01/2026	Fake news site detection	Working correctly
	THURSDAY	08/01/2026	Trusted source override logic	Added
	FRIDAY	09/01/2026	VirusTotal API integration	Security module added
	SATURDAY	10/01/2026	Malware scan testing	Functional
	SUNDAY	11/01/2026	Security + domain integration	Pipeline improved

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 6	TUESDAY	20/01/2026	Sentiment analysis module	TextBlob integrated
	WEDNESDAY	21/01/2026	Polarity & subjectivity analysis	Working
	THURSDAY	22/01/2026	Sentiment visualization	Graphs added
	FRIDAY	23/01/2026	Heuristic logic implementation	Basic rules added
	SATURDAY	24/01/2026	Testing sentiment module	Verified
	SUNDAY	25/01/2026	Optimization of analysis flow	Improved speed

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 7	TUESDAY	03/02/2026	Fact-checking module design	Planned
	WEDNESDAY	04/02/2026	Fact-check API integration	Working
	THURSDAY	05/02/2026	Cross-check claim logic	Implemented
	FRIDAY	06/02/2026	Result summarization logic	Added
	SATURDAY	07/02/2026	Fact-check testing	Accurate
	SUNDAY	08/02/2026	Module integration with pipeline	Completed

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 8	TUESDAY	17/02/2026	UI enhancement	Dashboard improved
	WEDNESDAY	18/02/2026	Metrics & charts	Visualization added
	THURSDAY	19/02/2026	Progress indicators	User feedback improved
	FRIDAY	20/02/2026	Error handling	Added fallback logic
	SATURDAY	21/02/2026	Testing full system	Stable
	SUNDAY	22/02/2026	Bug fixing	Issues resolved

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 9	TUESDAY	17/03/2026	Full pipeline testing	Working end-to-end
	WEDNESDAY	18/03/2026	Performance analysis	Response time measured
	THURSDAY	19/03/2026	Accuracy evaluation	Results recorded
	FRIDAY	20/03/2026	Comparison with existing systems	Analysis completed
	SATURDAY	21/03/2026	Documentation writing	Started
	SUNDAY	22/03/2026	Report improvement	Updated

	Day	Date	Name of the Topic/Module Completed	Remarks
WEEK No 10	TUESDAY	24/03/2026	Final report preparation	Completed
	WEDNESDAY	25/03/2026	Project review & final testing	Ready for submission

Remarks	

Project Guide Name	Project Guide's Signature
Dr. Jasmeet Kaur Ghai	
Dr. Rashmi M. Pote	
Ms. Sunita B. Rai	

Seal of the
University / College